

Basilix Leafy AI

CYBERSECURITY REPORT

THASHMI SHENALI ERIYAGAMA

System version 1.0.0 • Reviewed 12 October 2026 • Evidence-based static

1 Executive summary

Basilix Leafy AI is an example of a smart farming platform which includes different technological pieces built into one system. In particular, it includes a front-end that was created using React, an API layer based on Node/Express, a farm engine developed in Python/FastAPI, and local AI based on retrieval-augmented generation (RAG). Storage consists of PostgreSQL and TimescaleDB.

Moreover, this system includes cameras, sensors, and controllers responsible for controlling physical devices involved in agricultural processes.

Regarding security measures, it uses the following: limited Google login, backend authentication, service-to-service signed tokens, HTTPS on the front end, security approvals, rate limiting, and log maintenance.

2 Assessment scope and method

Reviewed artifacts included an extensive list of the following components.

The structure includes Docker Compose and Nginx, Express back-end for authentication, authorization, and routing, FastAPI framework with security plugins, AI core with reasoning and retrieval augmented generation (RAG), database setup scripts, camera and actuator drivers, front-end session handling code, and dependencies.

Methods used:

Targeted manual static code analysis was performed, including service trust boundary analysis, authorization analysis, physical control path analysis, and recovery configuration analysis. syntax validation was done for 51 Python files through AST parsing and for backend JavaScript files through Node's check flag. All the syntax validation for the code artifacts was successful. What should be pointed out is that no installation validation was needed, and no artifacts for vulnerable dependencies, credentials, physical devices, penetration test results, TLS scans, functional integration tests, or API verification were found.

Framework for classifying evidence:

"Observed." The behavior is confirmed by the source code or its configuration.

"Potential" represents a weakness that can potentially be exploited depending on specific conditions and/or environment configuration.

"Not observed" represents the fact that control was not present in the checked archive.

2.1 System architecture and trust boundaries

Function	System Architecture	Security controls

User access	React/Vite frontend → Nginx HTTPS → Express API	Google identity, session token, RBAC and feature permissions
Farm operations	Express API → FastAPI engine → schedules, sensors, cameras, KP303 powerboards	Service token plus backend user context; physical-actuator safety
AI operations	Farm engine → local Ollama AI core → restricted tool API / RAG	AI instruction and tool boundaries; recommendation approval

3 Implemented Security controls

Security Controls	Implementations	Implementation Status / Description
Google sign-in and account allowlist	webapp/backend/src/services/googleAuth.js; allowedUserAuth.js; routes/auth.js	Implemented. Verifies Google ID token/audience and verified email; allowlist checked.
RBAC and feature permissions	middleware/authorizeRole.js; services/accessControl.js; routes/safety.js; routes/farm.js	Implemented at backend. Role rechecked against enabled users for protected routes.
Application sessions	services/jwtAuth.js; sessionService.js; routes/authSession.js	Implemented. 1-hour signed access tokens, hashed rotating refresh tokens, HttpOnly/SameSite cookies.
Service authentication	services/engineAuth.js; engine/security/engine_auth.py; ai_core/llm_auth.py	Implemented. Ed25519 signed JWT with audience/issuer and short expiration.
Encrypted browser edge	webapp/nginx/nginx.conf; docker-compose.yml	Implemented. HTTP → HTTPS redirect and TLS 1.2/1.3; local CA deployment model.
Rate limiting	routes/auth.js; engine/api/rate_limit.py; engine_api.py	Implemented on Google login and many engine routes; complete coverage not established.
AI approval and risk defaults	engine/security/risk_checker.py; approval_manager.py; database/init.sql	Implemented. HIGH-risk recommendations require approval; unknown risk defaults HIGH.

Security Controls	Implementations	Implementation Status / Description
Dosing safeguards	engine/hal/farm_controls.py	Implemented. Numeric target bounds, sensor freshness, dose limits and pump-off checks.
Audit/alerts	routes/auth.js; logger/logger.py; engine/security/security_monitor.py	Implemented for selected events and sensor thresholds; coverage/retention need work.
Image path and size checks	engine/hal/ai_vision/detector.py	Implemented. Constrains local image root, type, size/pixels, strips metadata.

4 Findings and vulnerabilities

Ratings are qualitative (High / Medium / Low). A High finding may have strong operational impact even if exploitation requires prior access. No weakness below is described as a confirmed compromise.

Finding Id	Risk Severity	Identified Vulnerability / security Risk
F01	High	Shared database superuser-like credential across application components
F02	High	AI emergency stop is not a physical equipment shutdown
F03	High	User authority not cryptographically bound to internal service identity
F04	Medium	Internal HTTP and broad shared container network
F05	Medium	Manual equipment operations have limited audit visibility
F06	Medium	AI prompt-injection and over-trusting retrieved context
F07	Medium	Disaster recovery and security monitoring not evidenced
F08	Low–Medium	Request/resource abuse and token replay safeguards could be strengthened

F01 | High | Shared database superuser-like credential across application components

Evidence status: Design/Deployment dependent impact

• Source: docker-compose.yml, database/Dockerfile

Observation. Superuser credentials of the PostgreSQL database based on DB_USER and DB_PASSWORD are used by all four services–the initializer, engine, ingestion process, and web backend. The initial POSTGRES_USER always represents a superuser in PostgreSQL and requires confirmation of its real access rights that will allow enforcement of adequate confinement.

Impact. In case of compromise of one of the listed application services, an attacker will have the ability to read or change sensitive information in the same database cluster, such as farms settings, accounts and session data, recommendations, and logging data.

Remediation. Create unique roles without superuser privileges for engine, backend, and ingestion processes. Utilize migration and administrative credentials only during the deployment stage. Implement explicit permission controls on the level of tables and schemas. Do not use superuser credentials in applications and rotate them periodically.

Validation. Review the catalogs pg_roles and pg_auth_members. Each application account must not have rights to modify security tables and data of another application service

F02 | High | AI emergency stop is not a physical equipment shutdown

Status of Evidence: Functional restrictions identified.

Components referenced include: engine/managers/emergency_manager.py, engine/api/engine_api.py, and engine/hal/farm_controls.py.

Observation. Currently, the component under review has emergency_stop set to True and ai_enabled set to False. In the current execution path being reviewed, there is no call for stopping and powering

off irrigation, lights, fans, dosing pumps, or anything else. There is also no check for the ai_enabled flag anywhere in endpoints or in the manual control methods.

Risk. A person might expect that activating the “emergency stop” button will make sure that all equipment is stopped safely, but pumps and other equipment can still work in some cases. This is a design flaw in regards to safety and not the proof that the AI isolation control is broken.

Solution. The current interface needs to be renamed as “Emergency AI Isolation” or, even better, implement another independently developed hardware-safe-stop or interlock. An additional actuator status, fail-safe default values, and manual override are also required. Web UI alone can not be considered a certified safety system.

Validation.

AI isolation needs to be validated independently from the physical stopping of equipment by using actual equipment in the following situations: power loss, network loss, relay failure, and controller reboot.

F03 | High | User authority not cryptographically bound to internal service identity

Evidence status: Conditional exploitation of trust boundary.

Source: engine/api/engine_api.py; backend/services/engineClient.js.

Observation.

The user identity and role is derived from other headers: X-User-ID and X-User-Role. In some cases, the Engine applies generic access controls at higher level, allowing only OPERATOR and ADMIN roles, followed by finer grained access control on the backend side.

Risk.

Compromise of trusted backend or disclosure of signing key used for service signing will allow an attacker to create user context that will allow executing dangerous commands and/or cause incorrect logging of audit entries. It is not known if regular web browser user is able to bypass the backend using this approach.

Remediation.

Utilize time-limited signed delegation token that will include identity, permissions, method, path, audience, and expire claims. Delegation tokens need to be checked by the Engine against privileged actions (dosing, approval, emergency clearance).

Validation.

Create an internal caller with valid service credentials but fake role/header, check that operation will be denied and that audit logging is done properly.

F04 | Medium | Internal HTTP and broad shared container network

Evidence Status: The evidence is derived from the configuration and is subject to variations in the specific deployment.

Evidence-related data may be found at the following locations: docker-compose.yml, services/engineClient.js, and ai_core/llm_api.py.

Observation:

With the current configuration, communication between the Backend and Engine, as well as between the Engine and the AI services, takes place using the http:// scheme on the leafy-network Docker bridge that connects all services on one common network. While the use of signed tokens allows for proper authentication, tokens do not offer encryption of the payload in the request/response message.

Risk:

An attack on the container network or the Docker host itself would offer an opportunity for interception of internal communications and lateral movement to other hosts.

Remediation:

Introduce network segmentation between the frontend, application, AI services, database, and camera/IoT networks. Apply least privilege access control policy allowing only required communication routes. Mutual TLS (mTLS) should be considered as the method of internal service communication and access to the Ollama host. Database port binding should be limited to localhost only.

Validation:

Perform complete ACL validation and packet encryption verification in an isolated staging environment..

F05 | Medium | Manual equipment operations have limited audit visibility

Status of the Evidence: There is a logging gap here.

Code snippets which lead to this finding can be found in engine/api/engine_api.py, engine/hal/farm_controls.py, and logger/logger.py.

Finding:

Manual control handler for lights, irrigation, fans, and dosing target doesn't have an audit_log in their endpoint. On the contrary, other modules generate approvals, logs for state changes, etc. It shows that logging exists in some places, but it isn't everywhere.

Risk:

The inconsistent logging could hinder tracing of commands sent to the device, parameters used and physical state changes associated with those.

Solution:

Create events which would be sent before and after every command sent by a control handler. These events should contain information about the user who sends it, the command itself, its parameters, destination, previous state, outcome, correlation id and any other relevant data. Logs and off-site transmission of logs should be secured.

Verification:

Log both approved and declined commands and then disconnect the device and see if the order is immutable and administrator alerts were generated.

F06 | Medium | AI prompt-injection and over-trusting retrieved context

Evidence status: Potential; no exploit validated.

Source: ai_core/main.py; tools/rag_tool/retrieval.py; tools/create_recommendations.py.

Observation.

Information about the task/context is provided to the model using system messages and tool usage with RAG-retrieved information. RAG search contains appropriate parameter validation and dangerous actions are gated, but there is no systematic red-teaming of malicious content, trust labelling of content and deterministic validation of business rules in all proposed courses of action.

Risk.

Malicious content of source documents, poisoned knowledge documents or sensors' descriptions may affect reasoning and recommendations. The risk is mitigated by the allowlisted toolset and approval.

Remediation.

Knowledge and telemetry should be treated as untrusted evidence; trust labelling of source content; strict validation of all tool parameters and actions in Engine; logging of document provenance; checking for prompt injection; human review for physical actions.

Validation.

Adversarial testing of RAG and context (instruction override, false citation, multiple tool uses, unsafe objectives).

F07 | Medium | Disaster recovery and security monitoring not evidenced

Status of Evidence: Potential; exploit not yet demonstrated.

References: ai_core/main.py, tools/rag_tool/retrieval.py, tools/create_recommendations.py

Findings:

Messages sent to the system provide task and context information. RAG data is included in the tool messages. RAG data is properly validated, and actions that have a risk of being HIGH require approvals. The archive contains no red-teaming of malicious documents, no content trust labels, and no deterministic validation of all recommendations generated according to business rules.

Risk:

Source text, knowledge documents, and sensor descriptions may be malicious, poisoned, or misleading, affecting reasoning and recommendation generation. Risk is mitigated by the use of approved list of tools, along with the approval process.

Recommendations:

Assume knowledge and telemetry are untrusted evidence. Validate all tool arguments and actions that the Engine performs. Track document provenance; test for prompt injection attacks. Verify any physical changes via humans.

Validation:

Test RAG and context adversarially, including instruction override, false citation, repeated tools execution, and abuse of dangerous output.

F08 | Low–Medium | Request/resource abuse and token replay safeguards could be strengthened

Status of the Evidence: Limitations and Gaps Noted
Source: engine/api/engine_api.py, ai_core/main.py, engine/security/engine_auth.py

Observation:
The AI tool batch list request model lacks an explicit maximum limit specification. There is a de facto limit of 10 model turns and also an API rate limit. The jti claim of the Service JWT has a validity period of 180 seconds. There is no indication of a replay cache during token validation.

Risk:
With compromised credentials for the service, there is a possibility of a replay attack within the 180 seconds token validity period. Further, increasing the request batch size can lead to resource contention on a trusted internal endpoint.

Mitigation Strategy:
Apply an explicit maximum request batch size and maximum input length, along with concurrency and time limits. Check the replay protection measure for the jti in critical write APIs. Look at the application of one-time nonces/idempotency keys to counter a replay attack.

Testing Strategy:
Load test with restricted batch sizes and try processing of duplicate signed internal requests.

6) Limitations and constraints

- The portal of the system has been set up for domain leafy-ai.local with use of local CA certificate. It must be mentioned that the trust to public domain, internet presence and certificate management of devices have not been verified in this configuration.

- There are several parts used by the architecture – Docker host, PostgreSQL, Ollama host, cameras, environmental sensors and power boards. In case any of them fails, it might negatively affect visibility and safe actuation.

- The so-called "emergency stop" is realized using AI-based software isolation and not by physical emergency-stop circuit certification.

- Predictive outcomes will depend on the precision and relevance of the sensor data, camera footage and RAG documentation and might include inaccuracies.

- Gemini analysis sends preprocessed images from cameras to third parties in case this feature is turned on. Launching the service requires special attention to be paid to permissions, privacy and data retention policy.

- This code reflects current desired behavior and operations. Still, runtime configuration, network topology, physical controls and processes can change the risks.

- Along with this report, there was no auditing of dependencies automatically, no live exploits test, no protocol capture, no device safety testing and no operational incident response performed.

7) Validation and acceptance checklist

Testing Area	Validation and Acceptance Criteria
Identity and RBAC	Revoked and downgraded users lose access; operator cannot dose or approve when configured admin-only.
Service impersonation	Forged X-User-Role/ID cannot obtain extra privileges even with an authenticated service caller.
Physical safety	AI isolation works; independent safe-stop actually removes/halts dangerous outputs and is tested during failures.
Network and TLS	Only required service paths allowed; externally reachable endpoints use trusted TLS with security headers.
Database access	Application accounts are not superusers and cannot read/write unrelated tables.

Testing Area	Validation and Acceptance Criteria
AI red-team	Prompt injection and malicious RAG documents cannot bypass approval/target guardrails.
Audit assurance	Every actuator command, rejection and emergency event is attributable, time-stamped and retained off-host.
Backup and recovery	Full restoration of backend and operational database succeeds within agreed recovery objectives.
CI/CD and dependencies	Automated scans/tests run on changes; documented process exists for remediating High/Critical findings.

8 Conclusion

Basilix Leafy AI is an example of a product that was engineered using security engineering methods, which include authenticated Google identity, restricted user permissions, signed service messages, protection of AI actions, dosing, and operational logging.

The main steps after this include ensuring safe system operations during AI shutdown, restricting privilege sharing, protecting the trust boundary between the backend and the Farm Engine, and protecting operational outputs.

Appendix A – Security Testing Evidence

The following screenshots document separately conducted security checks and code inspections. Screenshots from earlier working copies are identified as historical evidence and must not be treated as proof that the same findings remain in version 1.0.0. A PASS applies only to the specific test shown, not the entire system.

Figure A1. engine requirements – Python dependency audit

PASS (limited scope) – pip-audit returned "No known vulnerabilities found" when resolving the version 1.0.0 engine requirements. Unpinned requirements may resolve to versions different from the deployed environment.



Figure A2. Invalid bearer token – authentication protection

PASS (runtime test) – A request to /api/auth/me using an invalid bearer token returned HTTP 401, indicating the tested endpoint rejected invalid authentication.

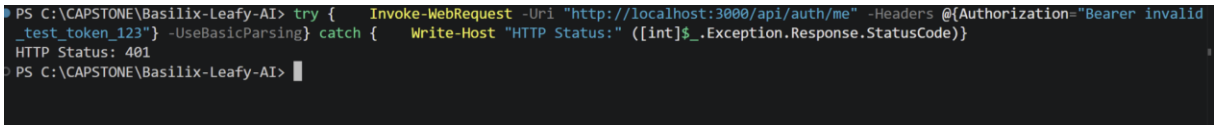


Figure A3. Login rate limiting – repeated invalid credentials

PASS (runtime test) – Ten unsuccessful Google login attempts returned HTTP 401; subsequent requests returned HTTP 429, showing the tested rate limit was enforced.

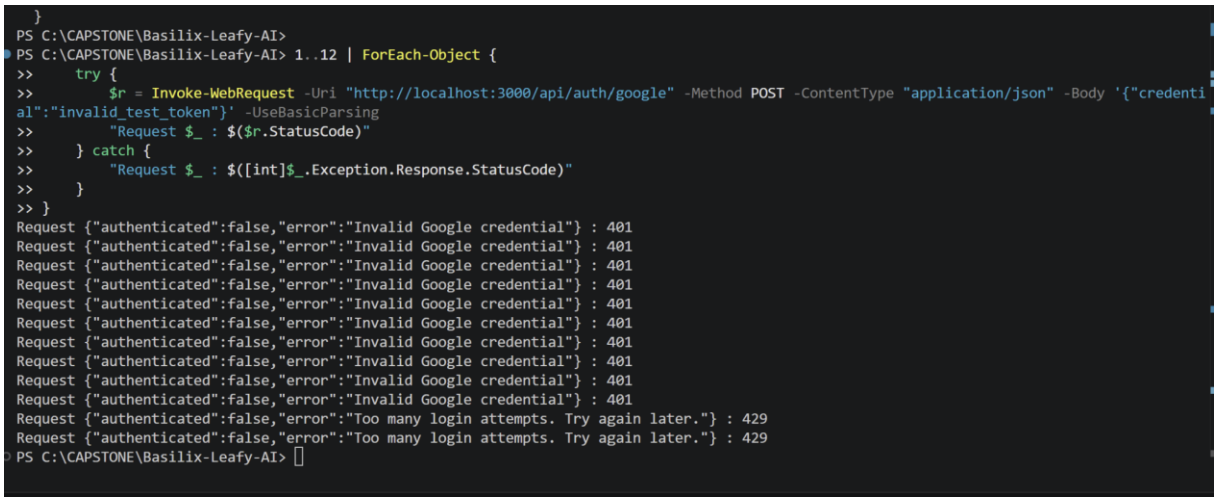


Figure A7. frontend and backend dependency audits

PASS (dependency audit) – npm audit reported 0 vulnerabilities in both webapp/backend and webapp/frontend in the tested version 1.0.0 working copy. This result covers known advisories reported for the audited dependency trees at the time of testing.

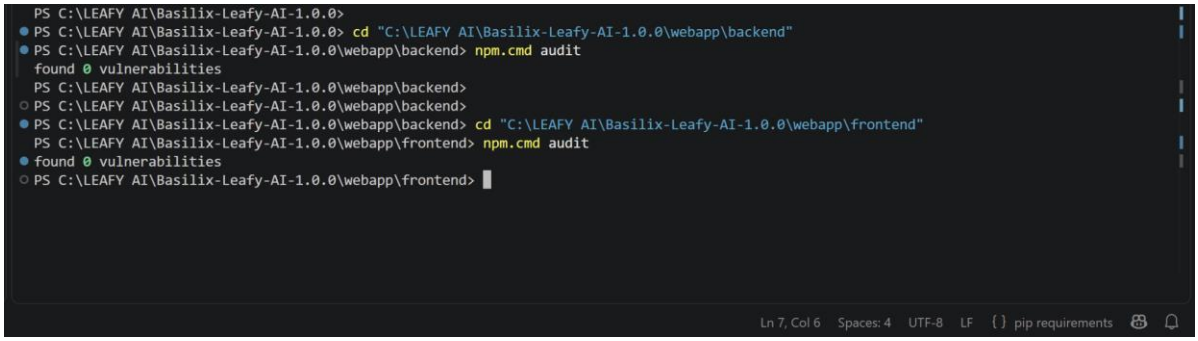


Figure A8. AI Core and AI Vision Python dependency audits

PASS (limited scope) – pip-audit reported no known vulnerabilities for leafy-ai/ai_core/requirements.txt and leafy-ai/engine/hal/ai_vision/requirements.txt. These requirement-based scans do not establish that every installed or deployed Python package was assessed.



References

ASVS/5.0/en/0x22-V13-Configuration.md at master · OWASP/ASVS. (n.d.). GitHub. https://github.com/OWASP/ASVS/blob/master/5.0/en/0x22-V13-Configuration.md

ASVS/5.0/en/0x25-V16-Security-Logging-and-Error-Handling.md at master · OWASP/ASVS. (n.d.). GitHub. https://github.com/OWASP/ASVS/blob/master/5.0/en/0x25-V16-Security-Logging-and-Error-Handling.md

Stouffer, K., Pease, M., Tang, C., Zimmerman, T., Pillitteri, V., Lightman, S., Hahn, A., Saravia, S., Sherule, A., Thompson, M., Computer Security Division, Information Technology Laboratory, Smart Connected Systems Division, Communications Technology Laboratory, & The MITRE Corporation. (2023). *NIST SP 800-82R3 Guide to Operational Technology (OT) Security*. U.S. Department of Commerce. https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-82r3.pdf